КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра програмних систем і технологій

Дисципліна

«КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»

Лабораторна робота № 3

«Захисне програмування, Обробка виключень та Модульне тестування.»

Виконав здобувач:	Роман ПЛАХОТА	**Перевірив**:	Павло ШАБАТІН
Група, підгрупа	ІПЗ-34, 10.	Дата перевірки	

2026

Варіант 5: Симулятор віртуальної файлової системи (VFS Shell)

Мета: Набути практичних навичок у:

- забезпеченні стійкості програмного забезпечення до некоректних вхідних даних;
- проєктуванні та реалізації ієрархії власних виключень (Custom Exceptions);
- написанні модульних тестів (Unit Tests) з використанням сучасних фреймворків;
- використанні Mock-об'єктів для ізоляції тестованих компонентів;
- вимірюванні та контролі покриття коду тестами (Code Coverage).

Умова завдання:

Задача у цій лабораторній роботі - додати захисне програмування. Програма повинна розпізнавати семантичні та синтаксичні помилки у вхідних файлах, перехоплювати їх та видавати користувачеві зрозумілі повідомлення про помилку замість аварійного завершення (Crash/Stacktrace). Також ви повинні написати Unit-тести, які автоматично перевірятимуть як коректні сценарії, так і сценарії з помилками.

Варіант 5: Симулятор віртуальної файлової системи (VFS Shell)

Приклади ситуацій для обробки та тестування:

- Конфлікт імен: Спроба створити папку/файл з іменем, яке вже існує в поточній директорії.
- Невалідний шлях: Спроба перейти (cd) у неіснуючу директорію або звернутися до файлу за невірним шляхом.
- Неприпустимі символи: Використання заборонених символів (наприклад, ?, *, \) у назві файлу.
- Порушення меж: Спроба видалити кореневу директорію /.

Хід роботи:

1) Ієрархія та оброблення помилок

Для того щоб реалізувати власне уніфіковане оброблення помилок та виведення відповідних повідомлень на екран, спочатку створимо власні класи помилок, що успадковуються від батьківського класу Exception

```python
class VFSBaseException(Exception):
    """Base exception for all VFS-related errors."""

    pass


class VFSSyntaxException(VFSBaseException):
    """Raised when the parser encounters invalid syntax or unknown commands."""

    pass


class VFSValidationException(VFSBaseException):
    """Raised when command arguments are invalid (e.g., wrong types or missing
arguments)."""

    pass


class VFSFileSystemException(VFSBaseException):
    """Raised when a file system operation fails (e.g., file not found, disk
full)."""

    pass


class VFSSecurityException(VFSBaseException):
    """Raised when a user lacks permissions to perform an action."""

    pass
```

Було реалізовано наступні помилки:

- VFSBaseException – базова помилка для всіх помилок пов’язаних з VFS системою та її правилами. Є батьківською помилкою для всіх більш прикладних помилок. Використання VFSBaseException як батьківського класу дозволяє визначати базову поведінку для всіх інших помилок, що є хорошим стилем відносно DRY принципу.
- VFSSyntaxException – піднімається коли парсер визначив неправильний синтакс або невідому команду
- VFSValidationException – піднімається коли аргументи для команди неправильні або відсутні
- VFSFileSystemException – піднімається коли якась операція відносно файла не вдалась(не знайдено файл, не вийшло створити файл і т.п.)
- VFSSecurityException – піднімається коли у користувача не достатньо прав для виконання бажаної задачі

Для забезпечення стабільності та інформативності програми, було впроваджено багаторівневу модель обробки помилок, яка базується на принципах «Fail Fast» та централізованого перехоплення.

Кожен модуль системи відповідає за перевірку власних інваріантів та генерацію специфічних для нього винятків:

- InputParser: Перевіряє коректність введених команд. Якщо назва команди відсутня у словнику фабрик, піднімається VFSSyntaxException. У випадку невідповідності типів аргументів (наприклад, замість розміру диска введено текст), виникає VFSValidationException.
- ExecutionEngine: Перед виконанням команди звертається до підсистеми безпеки. Якщо метод has_access повертає False, негайно генерується VFSSecurityException, що запобігає несанкціонованому доступу до даних.
- Бізнес-логіка команд: Під час виконання конкретних дій (наприклад, cd або mkdir) команди перевіряють стан файлової системи через контекст. Якщо шлях не існує або квоту пам'яті вичерпано, піднімається VFSFileSystemException.

Програма спроєктована таким чином, що проміжні шари не перехоплюють помилки всередині себе. Це дозволяє виняткам підніматись до найвищого рівня ієрархії — головного циклу програми. Такий підхід гарантує, що жодна помилка не буде проігнорована, а стан системи залишиться цілісним.

Обробка всіх винятків зосереджена в точці входу в програму(файл main.py) в методі для обробки команд. Завдяки тому, що всі кастомні класи успадковуються від VFSBaseException, ми можемо використовувати один блок except, щоб перехопити будь-яку специфічну помилку системи

```
try:
        # parse line into command
        command = parser.parse(line)

        # pass command to engine
        if command:
            engine.execute(command, line)

    except VFSBaseException as e:
        # Handling known application exceptions
        error_type = e.__class__.__name__
        formatted_error = f"[{error_type}] - {str(e)} at line {line_num}."
        formatter.render_error(formatted_error)
```

За виведення помилки користувачу на екран відповідає метод **render_error** в модулі OutputFormatter. Даний метод приймає в себе повідомлення у форматі [НАЗВА ПОМИЛКИ] – ПОВІДОМЛЕННЯ і виводить в термінал, наприклад:

```
vfs:/$ asdf
  [ERROR] [VFSSyntaxException] - Command 'asdf' is not supported at line 2.
vfs:/$
```

Як можемо побачити, помилка друкується в уніфікованому форматі, де вказано що це помилка, яка саме помилка, повідомлення, рядок на якому сталась помилка(корисно для script.sh) та відповідний колір.

2) Розробка юніт тестів

Для написання юніт тестів було обрано бібліотеку **pytest**, адже вона є однією з найбільш поширених бібліотек для написання тестів, а також має зручний функціонал для реалізації фікстур, мокування, різних видів assert і т.д.

Оскільки це юніт тести, підхід до тестування базувався на принципі ізоляції модулів: для перевірки логіки "ядра" використовувалися мок-об’єкти, що дозволило тестувати компоненти незалежно один від одного. За допомогою бібліотеки **coverage** було досягнуто показника покриття коду на рівні ~95%.

Було реалізовано наступні напрямки тестування для кожного модуля:

1) Models.py**:** Перевірялася логіка побудови ієрархічного дерева папок та файлів. Основна увага приділялася валідації вхідних даних на рівні конструкторів та методів маніпуляції деревом.
   Реалізовано негативні тести, що перевіряють реакцію системи на використання заборонених символів (?, *, \) у назвах. При спробі створення такого вузла модель викидає VFSValidationException.
   У разі спроби додати файл або папку з ім’ям, яке вже існує в поточній директорії, система піднімає VFSFileSystemException.
2) InputParser.py: Тестувалася коректність перетворення текстових рядків у відповідні об'єкти команд.

   Перевірка, чи створюються правильні екземпляри класів (MkdirCommand, CdCommand тощо) при валідних вхідних даних.

   Перевірка генерації VFSSyntaxException для невідомих команд та VFSValidationException для команд з некоректною кількістю або типом аргументів (наприклад, нечислове значення для розміру диска в mkfs)

3) ExecutionEngine.py: Для тестування цього модуля використовувалися мок-об’єкти через бібліотеку unittest.mock, щоб ізолювати Engine від реальної логіки команд та системи безпеки.

За допомогою мок-об’єктів SecurityEngine імітувалася ситуація відсутності прав доступу. Тест підтверджує, що в такому разі Engine зупиняє виконання та піднімає VFSSecurityException, не викликаючи метод execute самої команди.

4) Commands.py**:** Перевірялася безпосередня бізнес-логіка виконання операцій над файловою системою.
   Тестування команд cd, ls, cat на здатність коректно обробляти шляхи до об'єктів, яких не існує, або спроби доступу до файлу як до директорії. Очікувана реакція – VFSFileSystemException.
   Реалізовано тести на критичні операції, зокрема заборону видалення кореневої директорії / та перевірку квот (Quota Exceeded) при спробі створити файл, розмір якого перевищує доступний простір.
5) Context.py: Тестувався механізм управління глобальним станом віртуальної файлової системи, включаючи ініціалізацію та контроль ресурсів.
   Перевірялася логіка методів get_used_space та has_enough_space. За допомогою мок-об'єктів для кореневої директорії тестувалося, чи правильно контекст обчислює залишок вільного місця та чи блокує він операції, що перевищують ліміт max_size.
   Перевірка коректності ініціалізації: створення кореневого каталогу /, автоматичне призначення користувача admin та перевірка прапорця is_initialized залежно від переданого розміру диска.
6) OutputFormatter.py: Тестування візуального представлення даних та коректності форматування виводу в консоль. Оскільки модуль взаємодіє зі стандартним потоком виводу, використовувався фікстура capsys для перехоплення та аналізу результатів.
   Тестувалося, чи відповідають кольори виводу типу повідомлення (червоний для помилок, синій для тексту файлів, зелений для статусу [OK]).
7) SecurityEngine.py: Перевірка логіки розмежування прав доступу на основі рівнів доступу (POSIX-style) та ролей користувачів.
   Тестувався метод get_required_access, щоб переконатися, що кожній команді (ls, touch, cd) відповідає правильний біт доступу (READ, WRITE або EXECUTE).
   Перевірка методу has_access для звичайних користувачів. Тестувалися ситуації, коли права доступу вузла (наприклад, 755 або 700) дозволяють або забороняють виконання конкретної дії через побітове порівняння.
   Окремий тест для перевірки правила "суперкористувача", де підтверджується, що користувач admin отримує доступ до будь-якого об'єкта незалежно від встановлених на ньому обмежень.

У результаті, всі юніт тести пройшли успішно, та тестами було покрито 95% коду.

Код програми:

Код програми даного застостунку надано в Github-репозиторії за посиланням: https://github.com/xanaxhol1c/vfs-shell-python

Результат роботи:

Обробка та виведення помилок в термінал:

```
vfs:/$ asd
  [ERROR] [VFSSyntaxException] - Command 'asd' is not supported at line 2.
```

```
vfs:/$ ls 1234

> ls 1234
  [ERROR] [VFSFileSystemException] - ls: cannot access '1234': No such file
or directory at line 4.
```

```
vfs:/$ touch
  [ERROR] [VFSValidationException] - Invalid arguments for 'touch': list ind
ex out of range at line 2.
```

```
vfs:/$ ls .
  [ERROR] [VFSSecurityException] - Access denied: user 'admin' lacks permiss
ions to execute 'ls' at line 3.
```

Юніт тести:

```
PS C:\Vscode\Repos\vfs-shell-python> poetry run coverage run -m pytest
============================= test session starts ==============================
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Vscode\Repos\vfs-shell-python
configfile: pyproject.toml
plugins: mock-3.15.1
collected 43 items

tests\test_commands.py ...............                                  [ 32%]
tests\test_context.py ......                                            [ 46%]
tests\test_engine.py ....                                               [ 55%]
tests\test_formatter.py ........                                        [ 74%]
tests\test_models.py ...                                                [ 81%]
tests\test_parser.py ...                                                [ 88%]
tests\test_security.py .....                                            [100%]

============================== 43 passed in 0.31s ==============================
PS C:\Vscode\Repos\vfs-shell-python>
```

Coverage report: 95%

Files | Functions | Classes

coverage.py v7.13.4, created at 2026-03-08 19:43 +0200

File ▲	statements	missing	excluded	coverage
src \ __init__.py	0	0	0	100%
src \ commands.py	118	14	0	88%
src \ context.py	13	0	0	100%
src \ engine.py	18	0	0	100%
src \ exceptions.py	10	0	0	100%
src \ formatter.py	30	0	0	100%
src \ models.py	44	11	0	75%
src \ parser.py	26	4	0	85%
src \ security.py	21	0	0	100%
src \ types.py	3	0	0	100%
tests \ __init__.py	0	0	0	100%
tests \ test_commands.py	89	0	0	100%
tests \ test_context.py	32	0	0	100%
tests \ test_engine.py	44	0	0	100%

Висновок:

У ході роботи впроваджено систему обробки помилок на базі ієрархії VFSBaseException, що забезпечує стабільність програми через принцип «Fail Fast» та централізоване перехоплення винятків у головному циклі. Кожен модуль системи самостійно валідує вхідні дані (права доступу, квоти, коректність шляхів), а результати виводяться в уніфікованому форматі через OutputFormatter, що гарантує стійкість застосунку до некоректних дій користувача.

Завдяки використанню фреймворку pytest та мок-об’єктів, було реалізовано ізольоване тестування бізнес-логіки без прямого маніпулювання файловою структурою. Охоплення широкого спектра сценаріїв – від перевірки бітових масок до валідації назв на заборонені символи – дозволило досягти показника

покриття коду на рівні 95%. Це підтверджує високу надійність розробленої архітектури та її повну відповідність технічним вимогам.